FINAL YEAR PROJECT MANAGEMENT PORTAL

RSCI

RSCI FYP Portal

REST API Specification

Final Year Project Management Portal
Riphah School of Computing & Innovation

VERSION
1.0

STATUS
Implementation baseline

DATE
14 October 2026

PROTOCOL
REST over HTTPS / JSON

Companion to SRS v1.0, SDD v1.0, and Database Design v1.0

DOCUMENT GOVERNANCE

Document control

Field	Value
Document	RSCI FYP Portal REST API Specification
Version / date	1.0 / 14 October 2026
Audience	Backend, frontend, mobile, QA, DevOps, security, data and integration teams
Companion specifications	RSCI FYP Portal SRS v1.0; SDD v1.0; Database Design v1.0
Governing academic input	RSCI FYP Framework v1.0
API status	Implementation baseline; policy values identified as configurable require institutional approval
Owner	Riphah School of Computing & Innovation (RSCI)
Change control	Changes require a versioned contract update, migration impact review, and QA regression scope

Interpretation rule Normative words SHALL, MUST, SHOULD, and MAY indicate mandatory, required, recommended, and optional behavior. Endpoint examples are illustrative contracts; IDs and timestamps are synthetic.

Approval record

Role	Name	Decision / date
FYP Coordinator	To be assigned	Pending
HOD / academic authority	To be assigned	Pending
Technical lead	To be assigned	Pending
Security reviewer	To be assigned	Pending

CONTENTS

Table of contents

Section	Pages
1. Introduction and authority	4-5
2. Protocol, response, and query conventions	6-9
3. Authentication, rate limits, and security	10-12
4. Endpoint reference	13-35
5. RBAC summary matrix	36-37
6. Event and webhook notes	38
7. Data, workflow, and implementation rules	39-42
8. Test contract and handover	43-44
Appendix A. Framework traceability	45-46
Appendix B. Error catalogue, changelog, open issues	47-49

Navigation Every endpoint row specifies method, path, authorized roles, input, success shape, and key errors. Full JSON examples are grouped with the workflow they clarify.

PURPOSE AND SCOPE

1. Introduction

1.1 Purpose

This specification defines the public application programming interface used by the RSCI FYP Portal clients and trusted integrations. It translates the portal's roles, entities, workflows, evaluation controls, meeting catalogue, evidence records, AI declarations, and reporting needs into an implementable HTTP contract.

1.2 Scope

The API covers identity; users; academic sessions and programs; project-type configuration; groups; proposals; supervision; meetings, attendance and effort; milestones, deliverables and submissions; AI-usage declarations; evaluations, panels, rubrics, scores and moderation; notifications; announcements; reports; and audit logs.

1.3 Out of scope for v1

- Payment processing, public applicant registration, plagiarism-engine internals, learning-management-system grade write-back, and native video conferencing.
- Direct database access by clients. All access is through authorized services.
- Hard-coded evaluation weights, group-size limits, supervision caps, milestone dates, or AI rules. These are configuration or catalogue data.

1.4 API style

REST over HTTPS with UTF-8 JSON is the default. Multipart form data is used only for file-bearing operations. Resources use plural nouns; workflow actions use explicit subresources or action suffixes.

AUTHORITY AND PRINCIPLES

1. Introduction

1.5 References and consistency

Reference	Use in this contract
RSCI FYP Portal SRS v1.0	Functional requirements, nine roles, workflow boundaries and quality attributes
RSCI FYP Portal SDD v1.0	Service boundaries, security architecture, integrations and deployment design
RSCI FYP Portal Database Design v1.0	Entity names, keys, constraints, audit and retention model
RSCI FYP Framework v1.0	Ten FYP types, common deliverables, category AI policies and 300-hour project expectation

1.6 Design principles

1. **Least privilege:** authorization evaluates role, academic session, program, group membership, allocation and evaluator assignment.
2. **Workflow integrity:** state changes use dedicated actions and transactional validation.
3. **Evidence first:** submissions, declarations, scores, sign-offs and audit records are immutable or versioned.
4. **Configuration over code:** academic policy changes do not require a deployment.
5. **Idempotent clients:** retry-safe writes use an Idempotency-Key where specified.
6. **Privacy by design:** responses expose only fields needed by the requesting role.

HTTP CONTRACT

2. Conventions

2.1 Base URL and versioning

```
https://fyp.rsci.edu.pk/api/v1
```

Placeholder host The hostname is a deployment placeholder and MUST be confirmed before production. The /api/v1 prefix is normative for this version.

Breaking changes require a new URL major version. Backward-compatible fields and endpoints MAY be added within v1. Deprecated operations return `Deprecation` and `Sunset` headers for an approved transition period.

2.2 Required headers

Header	Rule
Authorization	Bearer <access_token> for protected operations
Content-Type	application/json; charset=utf-8, or multipart/form-data
Accept	application/json unless a report requests another format
X-Request-Id	Optional caller correlation ID; server returns one in every response
Idempotency-Key	Required for retry-prone creates: submissions, scores, bulk import, panel assignment
If-Match	Recommended on mutable resources; value is the returned ETag

2.3 Identifiers and time

Identifiers are opaque UUID strings. Clients MUST NOT derive meaning from them. Dates and timestamps use ISO 8601; stored and returned timestamps are UTC with `Z`. Academic dates without time use `YYYY-MM-DD`.

RESPONSE SHAPES

2. Conventions

2.4 Standard response envelope

```
{
  "success": true,
  "data": { "id": "8df4f9e4-1f8b-4f8b-8e04-8f8940c2d0d0" },
  "error": null,
  "meta": { "request_id": "req_01K7..." }
}
```

```
{
  "success": false,
  "data": null,
  "error": {
    "code": "VAL_001",
    "message": "Validation failed",
    "details": [{"field":"title","reason":"required"}]
  },
  "meta": { "request_id": "req_01K7..." }
}
```

2.5 Collection envelope

```
{
  "success": true,
  "data": [{"id":"...","name":"..."}],
  "error": null,
  "meta": {"page":1,"per_page":20,"total":73,"total_pages":4}
}
```

`meta` MAY also include filters applied, generated_at, or report warnings. Unknown response fields must be ignored by clients.

QUERY AND FILE RULES

2. Conventions

2.6 Pagination, filtering, sorting, and search

Convention	Contract
Pagination	page is 1-based; per_page defaults to 20 and MUST be 1-100
Filtering	Exact filters use field=value; multiple values use comma-separated values where documented
Ranges	Use from/to pairs, e.g. scheduled_from and scheduled_to
Sorting	sort=field or sort=-field; unsupported fields return VAL_001
Search	search is trimmed, case-insensitive, and limited to approved fields
Sparse fields	fields=id,title,status may be supported on high-volume reads; unauthorized fields remain hidden
Expansion	include=members,supervisor may be used only where documented

2.7 File uploads

File-bearing endpoints use `multipart/form-data`. The service validates extension, MIME type, signature, maximum size, malware-scan state, and authorization before making a file retrievable.

```
{
  "success": true,
  "data": {
    "file_id": "f_7c2...",
    "file_url": "https://fyp.rsci.edu.pk/files/signed/...",
    "file_name": "proposal-v2.pdf",
    "size_bytes": 483120,
    "scan_status": "CLEAN"
  }, "error": null
}
```

File URLs Returned file_url values SHOULD be short-lived signed URLs. Permanent storage keys are never exposed.

STATUS AND CONCURRENCY

2. Conventions

2.8 HTTP status usage

Status	Use
200 OK	Successful read or update
201 Created	Resource created; Location header returned
202 Accepted	Bulk import/report generation queued
204 No Content	Successful delete or action with no body
400 Bad Request	Malformed request or validation error
401 Unauthorized	Missing, invalid, or expired authentication
403 Forbidden	Authenticated but insufficient scope
404 Not Found	Resource absent or deliberately concealed
409 Conflict	Duplicate, stale state, or illegal transition
412 Precondition Failed	If-Match / ETag mismatch
413 Payload Too Large	Upload exceeds configured limit
422 Unprocessable Content	Semantically invalid bulk or scoring payload
429 Too Many Requests	Rate limit exceeded
500 / 503	Unexpected failure / temporarily unavailable dependency

2.9 Write semantics

- PATCH uses JSON Merge Patch unless an endpoint defines a command payload.
- DELETE is soft-delete for catalogue/configuration records already referenced; otherwise 204.
- Idempotency keys are scoped to caller + method + path and retained for a configurable period.
- Concurrent updates use ETags or a `version` field; stale writes return 412.

IDENTITY AND TOKENS

3. Authentication

3.1 Token model

Login returns a short-lived JWT access token and a rotating refresh token. Access tokens include user ID, role claims, session/program scopes, token ID, issuer, audience, issued time, and expiry. Refresh tokens are stored only as salted hashes, rotated on every use, and revoked as a family on reuse detection.

3.2 Authentication endpoints

Operation	Authorized roles	Parameters / payload	Success	Key errors
POST `/auth/login`	Public	Body: email, password, device_name?	200: tokens + user	AUTH_001, AUTH_004, RATE_001
POST `/auth/refresh`	Refresh token holder	Body: refresh_token	200: rotated token pair	AUTH_002, AUTH_005
POST `/auth/logout`	Authenticated	Body: refresh_token?; all_devices?	204	AUTH_002
GET `/auth/me`	Any authenticated role	None	200: effective profile, roles, scopes	AUTH_002
POST `/auth/password-change`	Any authenticated role	Body: current_password, new_password, logout_other_devices?	204	AUTH_001, VAL_001

3.3 Password and session controls

- Password policy, failed-attempt lockout, MFA eligibility, access-token lifetime, and refresh lifetime are environment configuration.
- Password change revokes prior refresh-token families except the current family when explicitly retained.
- Disabled users and ended assignments fail authorization immediately even if a cached token remains structurally valid.

LOGIN AND AUTHORIZATION

3. Authentication

3.4 Login response example

```
POST /auth/login
{
  "email": "student@example.edu.pk",
  "password": "••••••••",
  "device_name": "Portal web"
}

200 OK
{
  "success": true,
  "data": {
    "token_type": "Bearer",
    "access_token": "eyJ...",
    "expires_in": 900,
    "refresh_token": "rt_...",
    "refresh_expires_in": 2592000,
    "user": {
      "id": "0ec7b0d0-5b13-4b4d-a9ec-6a0a6b5db92d",
      "display_name": "Portal Student",
      "roles": ["STUDENT"],
      "scopes": {"session_ids":["ses_2026"],"program_ids":["prog_bsse"]}
    }
  }, "error": null,
  "meta": {"request_id":"req_01K7A"}
}
```

3.5 Authorization evaluation order

1. Validate token signature, issuer, audience, expiry, token ID, and revocation status.
2. Load active user and role assignments.
3. Verify endpoint role.
4. Verify resource scope: session, program, group membership, supervisor allocation, panel assignment, or partner association.
5. Verify workflow state and field-level permissions.
6. Write an audit event for sensitive reads and all mutations.

ABUSE PREVENTION

3. Security and rate limits

3.6 Proposed rate-limit policy

Class	Proposed configurable limit	Response
Login / refresh	10 attempts per 15 minutes per account + IP	429; Retry-After; security event
General reads	600 requests per 5 minutes per user	429; Retry-After
General writes	120 requests per 5 minutes per user	429; Retry-After
Uploads	20 files per 10 minutes per user	429 or 413
Reports / exports	10 generated exports per hour per user	429; existing job reference
Unauthenticated health	Infrastructure allowlist; separate policy	429 or 403

Values above are starting defaults, not academic policy. Limits SHALL be configurable and observable.

3.7 Security requirements

- TLS 1.2+; HSTS; secure, same-site cookies if tokens are delivered to a browser cookie.
- Parameterized queries; schema validation; HTML sanitization; MIME/signature verification; malware scanning.
- CSRF protection when cookie authentication is used. CORS restricted to approved origins.
- No tokens, passwords, declaration text, student reports, score narratives, or signed file URLs in logs.
- External examiner and industry partner access MUST be assignment-bound and time-limited.

USER LIFECYCLE

4.1 Users

4.1 User administration endpoints

Operation	Authorized roles	Parameters / payload	Success	Key errors
GET `/users`	SA, FC, HOD	Query: role, session, program, active, search, page, per_page, sort	200: paged user summaries	RBAC_001, VAL_001
POST `/users`	SA, FC	Body: institutional_id, email, display_name, roles[], program_ids[], active	201: user	VAL_001, CONFLICT_001
GET `/users/{id}`	SA, FC, HOD; self limited	Path: id; Query: include=roles,assignments	200: user profile	NF_001, RBAC_001
PATCH `/users/{id}`	SA, FC; self limited fields	Path: id; Body: mutable profile + role scopes; If-Match	200: updated user	VAL_001, RBAC_001, PRECOND_001
POST `/users/bulk-import`	SA, FC	Multipart: file CSV; dry_run?; Idempotency-Key	202: import job + validation summary URL	FILE_001, VAL_001, CONFLICT_001
PATCH `/users/{id}/activate`	SA, FC	Body: active, reason	200: user activation state	NF_001, CONFLICT_002

CSV import contract

Required headers: `institutional_id,email,display_name,role`. Optional headers: `program_code,session_code,phone`. Dry-run parses and validates without writes. A committed import is atomic per row, produces accepted/rejected counts, and exposes a downloadable rejection file.

ACADEMIC SESSION

4.2 Sessions and programs

4.2 Academic configuration endpoints

Operation	Authorized roles	Parameters / payload	Success	Key errors
GET `/sessions`	SA, FC, HOD, SUP, CO, STU, IE, EE, IP	Query: status, year, page, per_page	200: scoped sessions	VAL_001
POST `/sessions`	SA, FC	Body: code, name, start_date, end_date, status=DRAFT	201: session	VAL_001, CONFLICT_001
GET `/sessions/{id}`	Scoped authenticated	Path: id	200: session	NF_001, RBAC_001
PATCH `/sessions/{id}`	SA, FC	Body: name/dates/config; If-Match	200: session	WF_001, PRECOND_001
POST `/sessions/{id}/activate`	SA, FC	Body: confirmation, reason	200: status=ACTIVE	WF_001, VAL_001
GET `/sessions/{id}/programs`	Scoped authenticated	Query: active	200: program list	NF_001, RBAC_001
POST `/sessions/{id}/programs`	SA, FC	Body: program_id, group_size_min/max, supervision_cap?	201: session-program	CONFLICT_001, VAL_001
GET `/sessions/{id}/calendar`	Scoped authenticated	None	200: milestones + evaluation windows	NF_001
PUT `/sessions/{id}/calendar`	SA, FC	Body: timezone, events[]	200: calendar version	VAL_001, CONFLICT_002

Configuration boundary Group size, supervision cap, calendar windows, late rules, and evaluation weights are session-level configuration and require approval outside this API specification.

MULTI-DOMAIN CATALOGUE

4.3 Project-type catalogue

4.3 Project-type endpoints

Operation	Authorized roles	Parameters / payload	Success	Key errors
GET `/project-types`	All authenticated roles	Query: program, active, search	200: types + AI policy summary	VAL_001
POST `/project-types`	SA, FC	Body: code, name, suitable_programs[], description, outcomes[], ai_policy, active	201: project type	VAL_001, CONFLICT_001
GET `/project-types/{id}`	All authenticated roles	Path: id; include=deliverable_templates	200: type detail	NF_001
PATCH `/project-types/{id}`	SA, FC	Body: mutable catalogue fields; If-Match	200: type	PRECOND_001, WF_001
DELETE `/project-types/{id}`	SA	Path: id	204 or 200 archived	CONFLICT_002, RBAC_001

Catalogue resource minimum fields


```
{
  "id":"pt_product", "code":"PRODUCT_DEVELOPMENT",
  "name":"Product Development Project",
  "suitable_programs":["CS","SE","IT"],
  "ai_policy": {
    "core_functionality":"PROHIBITED",
    "documentation":"PROHIBITED",
    "api_based_ai_feature":"ALLOWED",
    "declaration_required":true
  },
  "active":true, "version":3
}
```


Framework fidelity The Appendix lists all ten framework categories. Exact policy text should be preserved in catalogue policy_notes alongside normalized enforcement fields.

GROUP FORMATION

4.4 Groups

4.4 Group endpoints

Operation	Authorized roles	Parameters / payload	Success	Key errors
POST `/groups`	STU; FC override	Body: session_id, program_id, project_type_id, name	201: group status=FORMING	VAL_001, CONFLICT_002
GET `/groups`	SA, FC, HOD, SUP, CO, STU, IE, EE, IP scoped	Query: session, program, type, status, supervisor_id, search	200: paged scoped groups	RBAC_001, VAL_001
GET `/groups/{id}`	Scoped roles	Path: id; include=members,proposal,allocations	200: group	NF_001, RBAC_001
PATCH `/groups/{id}`	Group leader in FORMING; FC	Body: name, project_type_id; If-Match	200: group	WF_001, PRECOND_001
POST `/groups/{id}/members`	Group leader; FC	Body: user_id, member_role	201: membership	CONFLICT_002, VAL_001
DELETE `/groups/{id}/members/{userId}`	Group leader; FC	Path IDs; Body: reason?	204	WF_001, CONFLICT_002
POST `/groups/{id}/join-requests`	STU	Body: message?	201: join request=PENDING	CONFLICT_001, WF_001
PATCH `/join-requests/{id}`	Group leader; FC	Body: decision=APPROVE|REJECT, comment?	200: request; membership if approved	WF_001, CONFLICT_002

Membership changes after proposal submission require coordinator permission and an auditable reason. Program compatibility, configured group-size limits, duplicate membership, and active session are validated transactionally.

PROPOSAL LIFECYCLE

4.5 Proposals

4.5 Proposal endpoints

Operation	Authorized roles	Parameters / payload	Success	Key errors
POST `/groups/{id}/proposal`	Group member; FC	Body: title, problem_statement, objectives[], scope, methodology, technologies[], industry_partner_id?	201: proposal=DRAFT	WF_001, VAL_001
GET `/proposals/{id}`	Scoped roles	Path: id; include=reviews,versions	200: proposal	NF_001, RBAC_001
PATCH `/proposals/{id}`	Group member in DRAFT/REVISION_REQUIRED; FC	Body: editable proposal fields; If-Match	200: proposal + version	WF_001, PRECOND_001
POST `/proposals/{id}/submit`	Group leader	Body: declaration_confirmed=true; Idempotency-Key	200: SUBMITTED + submitted_at	WF_001, VAL_001
POST `/proposals/{id}/reviews`	SUP, CO, FC, HOD by stage	Body: stage, decision, comments, conditions?	201: review + derived proposal status	RBAC_001, WF_001, VAL_001
GET `/proposals`	SA, FC, HOD, SUP, CO scoped	Query: status, session, program, type, reviewer_id, page	200: review queue	RBAC_001, VAL_001

Proposal state rules

DRAFT -> SUBMITTED -> UNDER_REVIEW -> APPROVED. A review may move UNDER_REVIEW to REVISION_REQUIRED or REJECTED. REVISION_REQUIRED can be edited and resubmitted. Only the coordinator/HOD policy may permit reopening APPROVED or REJECTED records.

WORKFLOW EXAMPLE

4.5 Proposals

Proposal submit and review example

```
POST /proposals/pr_102/submit
Idempotency-Key: 89d53f70-...
{"declaration_confirmed":true}

200 OK
{
  "success":true,
  "data":{"id":"pr_102","status":"SUBMITTED",
          "version":4,"submitted_at":"2026-10-14T09:15:00Z"},
  "error":null
}

POST /proposals/pr_102/reviews
{
  "stage":"COORDINATOR_REVIEW",
  "decision":"REVISION_REQUIRED",
  "comments":"Narrow objective 3 and add measurable acceptance criteria.",
  "conditions":["Revise scope boundary","Add validation sample"]
}

201 Created
{
  "success":true,
  "data":{"review_id":"rv_701","proposal_id":"pr_102",
          "decision":"REVISION_REQUIRED",
          "proposal_status":"REVISION_REQUIRED",
          "reviewed_at":"2026-10-14T11:20:00Z"},
  "error":null
}
```

Transactional effects

- Submit freezes that proposal version, validates group eligibility, and emits `proposal.submitted`.
- Review records are append-only. A correction creates a superseding review; it does not overwrite history.
- The state transition, audit event, and notification outbox write occur in one transaction.

SUPERVISOR ALLOCATION

4.6 Supervision

4.6 Faculty and allocation endpoints

Operation	Authorized roles	Parameters / payload	Success	Key errors
GET `/faculty`	SA, FC, HOD, STU limited	Query: expertise, program, available, load_min/max, search	200: faculty + active load	VAL_001, RBAC_001
POST `/allocations`	SA, FC, HOD	Body: group_id, supervisor_id, alloc_role=SUPERVISOR|CO_SUPERVISOR, start_date	201: allocation	CONFLICT_001, CONFLICT_002
GET `/allocations`	SA, FC, HOD, SUP, CO	Query: supervisor_id, group_id, role, active, session	200: scoped allocations	RBAC_001, VAL_001
PATCH `/allocations/{id}`	SA, FC, HOD	Body: supervisor_id? role? end_date? reason; If-Match	200: reassigned/ended allocation	WF_001, PRECOND_001

Allocation invariants

- A group has at most one active primary supervisor; co-supervisor count follows session policy.
- Load is calculated from active allocations, not a client-supplied value.
- Exceeding a configured cap is blocked unless an authorized override includes a reason.
- Reassignment ends the old allocation and creates a new history row; prior reviews and meetings remain attributed to their original actor.

MEETINGS AND LOGBOOK

4.7 Meetings

4.7 Meeting catalogue endpoints

Operation	Authorized roles	Parameters / payload	Success	Key errors
POST `/groups/{id}/meetings`	SUP, CO; FC	Body: scheduled_at, mode, venue_or_link, agenda, duration_minutes	201: meeting=SCHEDULED	VAL_001, CONFLICT_002
GET `/groups/{id}/meetings`	Group members, allocations, FC/HOD	Query: status, from, to, page	200: meetings	RBAC_001, VAL_001
PATCH `/meetings/{id}`	Meeting owner; SUP; CO; FC	Body: schedule/agenda/status; If-Match	200: meeting	WF_001, PRECOND_001
DELETE `/meetings/{id}`	Meeting owner; FC	Path: id; reason?	204 or cancelled record	WF_001, RBAC_001
POST `/meetings/{id}/attendance`	SUP, CO	Body: entries[{user_id,status,joined_at?,left_at?}]	200: attendance set	VAL_001, WF_001
POST `/meetings/{id}/minutes`	SUP, CO; assigned STU drafts	Body: notes, decisions[], action_items[], effort_hours_logged[]	201: minutes=DRAFT	VAL_001, WF_001
POST `/meetings/{id}/sign-off`	SUP; CO if delegated	Body: decision=APPROVE|RETURN, comment?	200: minutes status	WF_001, RBAC_001
GET `/groups/{id}/effort-summary`	Group members, SUP, CO, FC, HOD	Query: from?, to?, member_id?	200: target, totals, weekly trend	RBAC_001

Signed minutes are immutable. Corrections create an amendment linked to the signed version. Attendance and effort entries retain actor and timestamp provenance.

EFFORT TRACKING

4.7 Meetings

Effort summary example

```
GET /groups/gr_44/effort-summary?from=2026-08-01&to=2026-10-14

200 OK
{
  "success":true,
  "data":{
    "group_id":"gr_44",
    "target_hours":300,
    "total_hours":86.5,
    "progress_percent":28.83,
    "members":[
      {"user_id":"u_11","hours":44.0},
      {"user_id":"u_12","hours":42.5}
    ],
    "weekly_trend":[
      {"week_start":"2026-10-05","hours":17.0},
      {"week_start":"2026-10-12","hours":12.5}
    ],
    "last_signed_meeting_at":"2026-10-13T10:00:00Z",
    "risk_flags":["BELOW_WEEKLY_PLAN"]
  }, "error":null,
  "meta":{"generated_at":"2026-10-14T12:00:00Z"}
}
```

Effort rules

- The framework target is 300 hours for the complete project and references 9 hours per week. Whether tracking is per student or per group MUST be confirmed as an open policy decision.
- Only signed meeting minutes and approved standalone log entries count toward official totals.
- Negative hours are prohibited; maximum daily/weekly entries are configurable validation controls.
- Risk flags are advisory and never alter grades automatically.

MILESTONES AND CHECKLISTS

4.8 Milestones and templates

4.8 Milestone and deliverable-template endpoints

Operation	Authorized roles	Parameters / payload	Success	Key errors
GET `/sessions/{id}/milestones`	Scoped authenticated	Query: program, project_type, active	200: milestones	NF_001, RBAC_001
POST `/sessions/{id}/milestones`	SA, FC	Body: code, title, due_at, applies_to[], late_policy, sequence	201: milestone	VAL_001, CONFLICT_001
GET `/deliverable-templates`	All authenticated; scoped	Query: project_type_id, milestone_id, active	200: templates	VAL_001
POST `/deliverable-templates`	SA, FC	Body: code, title, instructions, file_rules, ai_declaration_required, applies_to[]	201: template	VAL_001, CONFLICT_001
GET `/deliverable-templates/{id}`	Scoped authenticated	Path: id	200: template	NF_001
PATCH `/deliverable-templates/{id}`	SA, FC	Body: mutable fields; If-Match	200: template	PRECOND_001, WF_001
DELETE `/deliverable-templates/{id}`	SA, FC	Path: id	204 or archived	CONFLICT_002
GET `/groups/{id}/deliverable-checklist`	Group members, allocations, FC/HOD	Query: status?	200: merged common + type-specific checklist	RBAC_001

Checklist generation snapshots the applicable template version when a group is approved. Later catalogue changes do not silently rewrite an in-progress group's obligations; coordinators may run an explicit migration.

DELIVERABLE EVIDENCE

4.9 Submissions

4.9 Submission endpoints

Operation	Authorized roles	Parameters / payload	Success	Key errors
POST `/groups/{id}/submissions`	Group member	Multipart: template_id, file, notes?, version_label?; Idempotency-Key	201: submission + file metadata	FILE_001, VAL_001, WF_001
GET `/groups/{id}/submissions`	Group members, allocations, evaluators, FC/HOD	Query: template_id, status, latest_only, page	200: scoped submissions	RBAC_001, VAL_001
PATCH `/submissions/{id}/review`	SUP, CO, FC, assigned evaluator by template	Body: decision=ACCEPT|RETURN|REJECT, comments, rubric_ref?	200: review + submission status	WF_001, RBAC_001, VAL_001

Upload and versioning behavior

- Every upload creates a new immutable submission version linked to the same checklist item.
- A checksum prevents accidental duplicate bytes but does not replace idempotency handling.
- Malware-pending files return 202 or a submission with `scan_status=PENDING`; reviewers cannot open them until clean.
- Late status is derived server-side from milestone due date, extension, and accepted server timestamp.
- Removal is an auditable withdrawal action; a referenced or reviewed file is never physically overwritten.

SUBMISSION CONTRACT EXAMPLE

4.9 Submissions

Submission and AI declaration example

```
POST /groups/gr_44/submissions  (multipart/form-data)
template_id=dt_srs
notes=Version for supervisor review
file=@RSCI_FYP_SRS_v1.pdf

201 Created
{
  "success":true,
  "data":{"id":"sub_310","template_id":"dt_srs","version":2,
    "status":"SUBMITTED","submitted_at":"2026-10-14T13:10:00Z",
    "file":{"file_id":"f_982","file_name":"RSCI_FYP_SRS_v1.pdf",
            "file_url":"https://fyp.rsci.edu.pk/files/signed/...",
            "scan_status":"CLEAN"}}, "error":null
}

POST /submissions/sub_310/ai-declaration
{
  "ai_usage_level":"NONE",
  "tools_used":[],
  "declaration_text":"No generative AI assistance was used in this document."
}

201 Created
{
  "success":true,
  "data":{"id":"aid_83","submission_id":"sub_310",
    "policy_flag":"COMPLIANT","attestation_status":"PENDING",
    "evaluated_against":{"project_type_id":"pt_product","policy_version":3}},
  "error":null
}
```

AI POLICY EVIDENCE

4.10 AI declarations

4.10 AI-declaration endpoints

Operation	Authorized roles	Parameters / payload	Success	Key errors
POST `/submissions/{id}/ai-declaration`	Submitting group member	Body: ai_usage_level, tools_used[], declaration_text, affected_sections[]?	201: declaration + policy_flag	VAL_001, AI_001, WF_001
PATCH `/ai-declarations/{id}/attest`	Group member(s); SUP verification if configured	Body: attested=true, statement_version, comment?	200: attestation status	WF_001, RBAC_001
GET `/submissions/{id}/ai-declaration`	Group, allocations, assigned evaluators, FC/HOD	Path: submission id	200: declaration + policy evaluation	NF_001, RBAC_001

Normalized values

Field	Allowed values / behavior
ai_usage_level	NONE, ASSISTED, EXTENSIVE, API_FEATURE_ONLY, OTHER
tools_used[]	Objects with tool_name, purpose, sections, optional version
policy_flag	COMPLIANT, REVIEW_REQUIRED, NON_COMPLIANT, NOT_EVALUATED
attestation_status	PENDING, ATTESTED, VERIFIED, DISPUTED
policy snapshot	project_type_id, policy_version, policy text hash, evaluated_at

The service flags policy mismatches; it does not infer misconduct or apply penalties. Human review is required. Declaration content and attestations are versioned and audited.

EVALUATION CONFIGURATION

4.11 Evaluation stages and rubrics

4.11 Evaluation-stage endpoints

Operation	Authorized roles	Parameters / payload	Success	Key errors
GET `/sessions/{id}/evaluation-stages`	Scoped authenticated	Query: program, project_type, active	200: ordered stages + weights visibility	RBAC_001
POST `/sessions/{id}/evaluation-stages`	SA, FC	Body: code, name, sequence, window, weight?, publish_rule	201: stage	VAL_001, CONFLICT_001
GET `/rubrics`	SA, FC, HOD, SUP, CO, IE, EE; STU published	Query: session, stage, project_type, active	200: rubric summaries	RBAC_001
POST `/rubrics`	SA, FC	Body: name, stage_id, applies_to[], scale, total_points, version	201: rubric=DRAFT	VAL_001, CONFLICT_001
GET `/rubrics/{id}`	Scoped roles	Path: id; include=criteria	200: rubric	NF_001, RBAC_001
PATCH `/rubrics/{id}`	SA, FC while DRAFT	Body: metadata; If-Match	200: rubric	WF_001, PRECOND_001
DELETE `/rubrics/{id}`	SA, FC	Path: id	204 or archive	CONFLICT_002
POST `/rubrics/{id}/criteria`	SA, FC while DRAFT	Body: code, title, description, max_points, weight?, descriptors[]	201: criterion	VAL_001, WF_001
PATCH `/rubrics/{id}/criteria/{criterionId}`	SA, FC while DRAFT	Body: criterion fields; If-Match	200: criterion	WF_001, PRECOND_001
DELETE `/rubrics/{id}/criteria/{criterionId}`	SA, FC while DRAFT	Path IDs	204	WF_001, CONFLICT_002

Published rubrics are immutable. Editing creates a new version. The API validates criterion totals and stage association before publication.

EVALUATION EXECUTION

4.12 Evaluations and panels

4.12 Scheduling and panel endpoints

Operation	Authorized roles	Parameters / payload	Success	Key errors
POST `/evaluations`	SA, FC	Body: group_id, stage_id, scheduled_at, duration_minutes, venue, rubric_id	201: evaluation=SCHEDULED	VAL_001, CONFLICT_002
GET `/evaluations/{id}`	Group members, allocations, assigned panel, FC/HOD	Path: id; include=panel,summary	200: scoped evaluation	NF_001, RBAC_001
POST `/evaluations/{id}/panels`	SA, FC	Body: members[{user_id,panel_role}], chair_id?; Idempotency-Key	200: panel assignments	VAL_001, CONFLICT_002
POST `/evaluations/{id}/scores`	IE, EE, SUP if stage allows	Body: evaluator_id? criteria_scores[], overall_comment, recommendation; Idempotency-Key	201: evaluator score sheet=SUBMITTED	SCORE_001, WF_001, RBAC_001
GET `/evaluations/{id}/score-summary`	FC, HOD; panel after submit; students only when published	Path: id	200: aggregate + completion	RBAC_001, WF_001
PATCH `/evaluations/{id}/moderate`	FC, HOD; designated chair	Body: action, adjusted_scores? reason, evidence_ref?	200: moderated summary	WF_001, SCORE_002
PATCH `/evaluations/{id}/publish`	FC, HOD	Body: publish=true, note?	200: PUBLISHED + timestamp	WF_001, CONFLICT_002
GET `/groups/{id}/results`	Group members, allocations, FC/HOD	Query: stage_id?; students receive PUBLISHED only	200: visible result set	RBAC_001

Evaluators may save drafts through PATCH on their score sheet in implementation, but only the submit action participates in aggregation. An evaluator cannot see another evaluator's item-level scores before submitting unless moderation policy explicitly permits it.

SCORING EXAMPLE

4.12 Evaluations and panels

Score submission example

```
POST /evaluations/ev_55/scores
Idempotency-Key: 046b2c74-...
{
  "criteria_scores":[
    {"criterion_id":"cr_problem","points":14,"max_points":15,"comment":"Well scoped."},
    {"criterion_id":"cr_method","points":24,"max_points":30,"comment":"Add threat mitigation."},
    {"criterion_id":"cr_demo","points":36,"max_points":40,"comment":"Stable demonstration."}
  ],
  "overall_comment":"Strong progress with a clear validation gap.",
  "recommendation":"PASS_WITH_ACTIONS"
}

201 Created
{
  "success":true,
  "data":{"score_sheet_id":"ss_88","status":"SUBMITTED",
    "raw_total":74,"max_total":85,"normalized_percent":87.06,
    "submitted_at":"2026-10-14T14:30:00Z"},
  "error":null
}
```

Scoring validation

- Each criterion appears once; points are within bounds; all mandatory comments are present.
- The rubric snapshot attached to the evaluation controls scoring, even if a new rubric version exists.
- Totals are calculated server-side using decimal arithmetic and the configured rounding rule.
- Submitted score sheets are locked; authorized correction creates a revised sheet with reason and linkage.

AGGREGATION EXAMPLE

4.12 Evaluations and panels

Score summary example

```
GET /evaluations/ev_55/score-summary

200 OK
{
  "success":true,
  "data":{
    "evaluation_id":"ev_55","stage_id":"stage_mid",
    "status":"SCORED","panel_size":3,"submitted_sheets":3,
    "aggregation_method":"MEAN_NORMALIZED_PERCENT",
    "raw_aggregate_percent":84.72,
    "moderation":{"status":"NOT_REQUIRED","adjustment":0},
    "final_percent":84.72,
    "criterion_summary":[
      {"criterion_id":"cr_problem","mean_points":13.33,"max_points":15},
      {"criterion_id":"cr_method","mean_points":24.67,"max_points":30},
      {"criterion_id":"cr_demo","mean_points":34.00,"max_points":40}
    ],
    "publication":{"status":"UNPUBLISHED","published_at":null}
  }, "error":null
}
```

Moderation safeguards

Moderation requires an allowed action, reason, actor authority, and before/after values. Large variance thresholds, aggregation method, tie handling, rounding, and adjustment limits are configurable academic policy. Publication validates panel completion, moderation status, result completeness, and any hold flags.

MESSAGING

4.13 Notifications and announcements

4.13 Notification endpoints

Operation	Authorized roles	Parameters / payload	Success	Key errors
GET `/notifications`	Any authenticated role	Query: unread, type, from, page, per_page	200: recipient notifications	VAL_001
PATCH `/notifications/{id}/read`	Notification recipient	Body: read=true	200: read_at	NF_001, RBAC_001
POST `/notifications/read-all`	Any authenticated role	Body: before? type?	200: updated_count	VAL_001
GET `/announcements`	Scoped authenticated	Query: session, audience, active, page	200: announcements	RBAC_001
POST `/announcements`	SA, FC, HOD	Body: title, body, audience, session_id, publish_at, expire_at?	201: announcement	VAL_001, RBAC_001
GET `/announcements/{id}`	Scoped authenticated	Path: id	200: announcement	NF_001, RBAC_001
PATCH `/announcements/{id}`	Author; SA, FC	Body: mutable fields; If-Match	200: announcement	PRECOND_001, WF_001
DELETE `/announcements/{id}`	Author; SA, FC	Path: id	204	RBAC_001, WF_001

In-app notification delivery is authoritative. Email, push, and messaging channels are best-effort derivatives. Notification templates use versioned variables and MUST escape untrusted content.

REPORTING

4.14 Reports

4.14 Report endpoints

Operation	Authorized roles	Parameters / payload	Success	Key errors
GET `/reports/groups-summary`	SA, FC, HOD; SUP scoped	Query: session, program?, type?, status?, format=json|csv|pdf	200 file/JSON or 202 job	RBAC_001, VAL_001
GET `/reports/evaluation-progress`	SA, FC, HOD	Query: session, stage?, program?, format	200/202: schedule + scoring completion	RBAC_001
GET `/reports/at-risk-groups`	SA, FC, HOD; SUP scoped	Query: session, risk_type?, threshold_profile?, format	200/202: advisory risk list	RBAC_001, VAL_001
GET `/reports/effort-hours`	SA, FC, HOD; SUP scoped	Query: session, group_id?, supervisor_id?, from?, to?, format	200/202: approved effort	RBAC_001
GET `/reports/supervisor-load`	SA, FC, HOD	Query: session, program?, active?, format	200/202: allocation load	RBAC_001

Export behavior

- JSON may return synchronously within configured cost limits. CSV/PDF or large queries return 202 with a report job.
- Generated files use signed, expiring download URLs and inherit row-level authorization.
- Every report includes generated_at, filters, session, actor, and data classification.
- Risk reports disclose the factors used. They are advisory and require human review.

ACCOUNTABILITY

4.15 Audit

4.15 Audit endpoint

Operation	Authorized roles	Parameters / payload	Success	Key errors
GET `/audit-logs`	SA; FC scoped; HOD read	Query: entity_type, entity_id, actor_id, action, date_from, date_to, ip?, page, per_page	200: immutable audit records	RBAC_001, VAL_001

Audit record schema

```
{
  "id":"aud_991",
  "occurred_at":"2026-10-14T14:30:00Z",
  "actor":{"user_id":"u_90","role":"INTERNAL_EVALUATOR"},
  "action":"EVALUATION_SCORE_SUBMITTED",
  "entity":{"type":"score_sheet","id":"ss_88"},
  "context":{"request_id":"req_01K7A","ip_masked":"203.0.113.x"},
  "changes":{"before_hash":"...","after_hash":"..."},
  "reason":null
}
```

Audit controls

- Audit events are append-only, integrity-protected, timestamped, and retained according to approved policy.
- Sensitive values are redacted; password/token material and file content never appear.
- Audit export itself is audited. External examiners, students, supervisors, and partners cannot query the global log.
- Operational logs and academic audit records are separate stores with distinct retention.

CLIENT SUPPORT

4.16 Supporting reads

4.16 Recommended supporting endpoints

The following reads are needed for practical clients even though they are not the main workflow commands.

Operation	Authorized roles	Parameters / payload	Success	Key errors
GET `/programs`	Authenticated scoped	Query: active, search	200: program catalogue	VAL_001
GET `/lookups`	Authenticated	Query: keys=roles,statuses,decisions	200: permitted enum labels	VAL_001
GET `/files/{id}/download`	Authorized to owning entity	Path: file id	302 signed URL or 200 stream	FILE_002, RBAC_001
GET `/imports/{id}`	Import initiator; SA, FC	Path: import job id	200: status and counts	NF_001, RBAC_001
GET `/report-jobs/{id}`	Job initiator; SA, FC	Path: job id	200: status / download URL	NF_001, RBAC_001
GET `/health`	Infrastructure only	None	200: liveness	RATE_001
GET `/ready`	Infrastructure only	None	200/503 dependency readiness	RATE_001

Discovery rule

Clients SHALL use catalogue endpoints and response links rather than hard-coding database IDs. Enumeration codes may be cached by version; display labels remain server-driven for future localization.

CROSS-CUTTING RULES

4.17 Endpoint behavior summary

4.17 Common validation by domain

Domain	Mandatory server checks
Group / proposal	Active session; compatible program/type; membership; size policy; legal state
Allocation	Faculty active; program scope; role uniqueness; cap/override; date overlap
Meeting / effort	Group scope; actor allocation; plausible duration; sign-off; duplicate entry
Submission	Checklist item; deadline/extension; version; file policy; scan; declaration rule
Evaluation	Stage window; approved group; rubric snapshot; panel eligibility; schedule collision
Scores	Assigned evaluator; one active sheet; all criteria; point bounds; comment rules
Publication	Panel completeness; moderation; holds; stage prerequisites; result snapshot
Report	Session required; row-level scope; export budget; format; redaction

Idempotency matrix

Operation	Idempotency requirement
Proposal submit	Recommended
Submission upload	Required
Bulk import	Required
Panel assignment	Required
Score submit	Required
Notification read	Naturally idempotent
Report GET	Safe; asynchronous job may reuse canonical request hash

Duplicate idempotency keys with a different request body return `409 CONFLICT_003`. Replays with the same body return the original status and response.

SCHEMA CONTRACT

4.18 Resource schemas

4.18 Canonical resource fields

Resource	Required fields
User	id, institutional_id, email, display_name, active, roles[], version, created_at
Group	id, session_id, program_id, project_type_id, name, status, version
Proposal	id, group_id, title, problem_statement, objectives[], status, version
Allocation	id, group_id, supervisor_id, alloc_role, start_date, end_date?, active
Meeting	id, group_id, scheduled_at, status, agenda, owner_id, minutes_status
Milestone	id, session_id, code, title, due_at, sequence, applies_to[]
Submission	id, group_id, template_id, version, status, file, submitted_at
AI declaration	id, submission_id, usage_level, policy_flag, policy_snapshot, attestations
Evaluation	id, group_id, stage_id, rubric_version_id, schedule, status
Score sheet	id, evaluation_id, evaluator_id, criterion_scores[], total, status
Audit log	id, occurred_at, actor, action, entity, context, changes

All mutable resources include `version`, `created_at`, `created_by`, `updated_at`, and `updated_by` unless the actor is system-owned. Soft-deleted catalogue records also include `archived_at`.

ROLE DEFINITIONS

5. RBAC matrix

5.1 Roles and scope

Role	Scope principle
Super Admin (SA)	Platform-wide administration; cannot bypass immutable audit/evidence rules
FYP Coordinator (FC)	Academic-session administration, configuration, workflows, allocation, publication
HOD	Oversight, approval, moderation and reports within assigned programs
Supervisor (SUP)	Assigned groups: meetings, reviews, submissions and permitted assessments
Co-Supervisor (CO)	Assigned groups; delegated supervision actions; no implied primary authority
Student (STU)	Own profile and group; drafts, submissions, declarations, visible results
Internal Evaluator (IE)	Assigned evaluations and authorized evidence only
External Examiner (EE)	Assigned evaluations, restricted time window and evidence set
Industry Partner (IP)	Associated projects; client feedback/acceptance only where enabled

Legend

A administer; **W** write/act; **R** read; **S** scoped to assignment or own group; **P** published-only; **-** denied by default. Multiple letters combine.

AUTHORIZATION SUMMARY

5. RBAC matrix

Endpoint group	SA	FC	HOD	SUP	CO	STU	IE	EE	IP
Auth / own profile	W	W	W	W	W	W	W	W	W
Users	A	A	R	S	S	S	S	S	S
Sessions / programs / calendar	A	A	R	R	R	R	R	R	R
Project types / templates	A	A	R	R	R	R	R	R	R
Groups / membership	A	A	R	S	S	WS	S	S	S
Proposals	A	A	R	WS	WS	WS	RS	RS	RS
Allocations / faculty load	A	A	A	RS	RS	-	-	-	-
Meetings / attendance / minutes	A	A	R	WS	WS	RS	-	-	RS
Submissions / AI declarations	A	A	R	WS	WS	WS	RS	RS	RS
Rubrics / evaluation stages	A	A	R	R	R	P	R	R	P
Evaluations / scores	A	A	A	WS	RS	P	WS	WS	RS
Results / publication	A	A	A	RS	RS	P	RS	RS	P
Announcements	A	A	A	R	R	R	R	R	R
Reports	A	A	A	RS	RS	-	-	-	-
Audit logs	A	AS	R	-	-	-	-	-	-

Deny by default The matrix is a summary, not the authorization algorithm. Resource scope, workflow state, field permission, active assignment, and publication status are always re-evaluated.

ASYNCHRONOUS INTEGRATION

6. Events and webhooks

6.1 Domain events

The service SHOULD write transactional outbox events for reliable asynchronous notification and integration. Initial consumers are internal notification, report, search-index, and audit projection workers.

Event	Minimum payload
proposal.submitted / reviewed	event_id, occurred_at, proposal_id, group_id, status, actor_id
allocation.changed	allocation_id, group_id, supervisor_id, action
meeting.signed_off	meeting_id, group_id, minutes_version, effort_total
submission.created / reviewed	submission_id, group_id, template_id, status
ai_declaration.flagged	declaration_id, submission_id, policy_flag
evaluation.scheduled	evaluation_id, group_id, stage_id, scheduled_at
score_sheet.submitted	score_sheet_id, evaluation_id, evaluator_id
result.published	evaluation_id, group_id, published_at
announcement.published	announcement_id, audience, session_id

6.2 External webhooks - future

External webhook delivery is optional and not required for v1 launch. If enabled, subscriptions are allowlisted, HTTPS-only, secret-signed, retried with exponential backoff, and disabled after repeated failures. Payloads contain identifiers and event metadata, not full student evidence.

```
X-RSCI-Event: proposal.submitted
X-RSCI-Delivery: del_01K7A
X-RSCI-Signature: t=...,v1=HMAC_SHA256(...)
{"event_id":"evt_...","occurred_at":"2026-10-14T09:15:00Z","data":{"proposal_id":"pr_102"}}
```

PERSISTENCE SEMANTICS

7. Data and workflow rules

7.1 Core relationships

Parent	Relationship	Child / constraint
Academic session	1:N	Programs, milestones, evaluation stages, groups
Project type	1:N	Deliverable templates and policy versions
Group	1:N	Members, proposal versions, allocations, meetings, submissions, evaluations
Proposal	1:N	Versions and append-only reviews
Meeting	1:1 active	Minutes; 1:N attendance, actions, effort entries, amendments
Submission	N:1	Checklist item; 1:1 active AI declaration version chain
Evaluation	N:1	Stage and rubric snapshot; 1:N panel assignments and score sheets
Score sheet	1:N	Criterion scores; supersession chain for corrections

7.2 Transaction boundaries

- State change + audit event + outbox event commit together.
- File bytes may upload before metadata commit; orphan cleanup runs asynchronously.
- Score aggregation runs after all required sheets submit and is recomputed deterministically from immutable inputs.
- Bulk import isolates row failures but commits accepted rows in bounded batches; job summary is final only after all batches complete.

WORKFLOW INTEGRITY

7. Data and workflow rules

7.3 State machines

Resource	Allowed progression
Session	DRAFT -> ACTIVE -> CLOSED -> ARCHIVED
Group	FORMING -> READY -> ACTIVE -> COMPLETED; CANCELLED from authorized states
Proposal	DRAFT -> SUBMITTED -> UNDER_REVIEW -> APPROVED; UNDER_REVIEW -> REVISION_REQUIRED / REJECTED
Meeting	SCHEDULED -> HELD -> MINUTES_DRAFT -> SIGNED_OFF; SCHEDULED -> CANCELLED
Submission	SUBMITTED -> UNDER_REVIEW -> ACCEPTED / RETURNED / REJECTED; new version after RETURNED
Evaluation	DRAFT -> SCHEDULED -> IN_PROGRESS -> SCORED -> MODERATED -> PUBLISHED
Score sheet	DRAFT -> SUBMITTED -> LOCKED; correction creates SUPERSEDED + new sheet
AI declaration	PENDING -> ATTESTED -> VERIFIED; any review may mark DISPUTED

7.4 Illegal transitions

Illegal transitions return `409 WF_001` with `current_state`, `requested_action`, and `allowed_actions`. Clients MUST not infer availability only from local UI state; command availability may change with policy, assignment, or concurrent activity.

RELIABILITY

7. Operations

7.5 Observability and service objectives

Concern	Implementation requirement
Correlation	Every request receives request_id; propagate trace context to workers
Metrics	Latency, throughput, error rate, authorization denials, queue depth, scan latency, report duration
Audit vs logs	Separate academic audit records from diagnostic logs
Alerts	Sustained 5xx, authentication anomalies, queue backlog, file-scan failure, database saturation
Tracing	Trace API, storage, database, outbox and worker spans without sensitive payloads
Availability target	To be approved in SRS/operations policy; avoid inventing an SLA here
Performance	Target budgets are configurable release criteria; measure p50/p95/p99 by route class

7.6 Caching

Public catalogue-style reads may use ETags and short private caches. Personalized data, signed URLs, scores, declarations, and audit responses use `Cache-Control: no-store`. Authorization is never satisfied from an unscoped cache key.

7.7 Backups and recovery

Database backups, object-store versioning, restore drills, RPO/RTO, and retention periods require approved operational values. Recovery must preserve audit linkage, file checksums, versions, and publication state.

CONFIGURATION

7. Deployment and configuration

7.8 Environment configuration

Configuration family	Examples
Identity	issuer, audience, token lifetimes, lockout, MFA policy
Academic	session dates, group size, supervision cap, stage weights, rounding, moderation
Workflow	review sequence, reopen authority, publication prerequisites
Files	allowed MIME/extensions, size limits, scan timeout, signed-URL lifetime
Notifications	channels, templates, quiet hours, retry policy
Rate limits	route classes, per-user/IP windows, report/export quotas
Retention	audit, submissions, operational logs, generated reports
Integrations	mail, object storage, malware scanner, optional identity provider

7.9 Deployment checklist

1. Confirm production base URL and TLS certificates.
2. Apply schema migrations and seed ten project types plus common deliverables.
3. Create initial session, programs, policy versions, milestones, stages and rubrics.
4. Configure identity, refresh rotation, secure secrets and administrator bootstrap.
5. Verify object storage, antivirus, email and worker queues.
6. Run authorization, workflow, file, scoring, export, audit and restore tests.
7. Publish OpenAPI 3.1 contract and generated client compatibility report.

QA BASELINE

8. Test contract

8.1 Minimum acceptance scenarios

ID	Scenario	Expected result
API-A01	Valid login, refresh rotation, old refresh reuse	New pair issued; reuse revokes family and emits security event
API-A02	Student queries another unrelated group	404 or 403 with no sensitive data
API-A03	Two clients patch same proposal version	First succeeds; stale If-Match returns 412
API-A04	Duplicate submission retry with same key/body	Original 201 response replayed; one submission version
API-A05	Proposal submitted from DRAFT then reviewed	Legal states, immutable review, audit/outbox emitted
API-A06	Meeting minutes signed then edited	Direct edit blocked; amendment flow required
API-A07	AI declaration conflicts with type policy	policy_flag=REVIEW_REQUIRED/NON_COMPLIANT; no automatic penalty
API-A08	Evaluator scores outside criterion bound	422 SCORE_001; no partial score sheet
API-A09	Student requests unpublished result	Result omitted/forbidden; published result visible
API-A10	Large report requested	202 job; scoped expiring download when complete

VERIFICATION

8. Test contract

8.2 Contract and security test suites

- **Schema:** request/response validation, enum compatibility, unknown fields, nullable fields, date/time formatting.
- **Authorization:** every endpoint across all nine roles, positive and negative resource scopes, ended assignments, inactive users.
- **Workflow:** every legal and illegal transition, concurrent actions, reopen and correction paths.
- **Files:** MIME spoofing, oversized files, malware states, duplicate bytes, expired signed URLs, unauthorized download.
- **Evaluation:** criterion totals, decimals, rounding, missing evaluators, variance, moderation, publication and corrections.
- **Reliability:** idempotent retries, outbox failure, queue redelivery, partial bulk import, report job timeout.
- **Privacy:** field redaction, search leakage, report scoping, logs, cache keys and audit access.

8.3 Definition of API-ready

Implementation is ready only when OpenAPI 3.1 validates, examples pass contract tests, database constraints match resource rules, negative authorization coverage is complete, migrations are reversible, operational dashboards exist, restore testing succeeds, and unresolved policy decisions are configured or deliberately disabled.

RSCI FYP FRAMEWORK V1.0

Appendix A. Framework traceability

A.1 Project types 1-5

Project type	Programs	Primary deliverables	AI policy normalization
Product Development Project	CS, SE, IT	Solution, source, SRS, SDD, test plan/report, manuals, final report	Core/docs prohibited; AI via APIs allowed
Entrepreneurial Project	CS, SE, IT	MVP, business plan/canvas, market research, financial feasibility, pitch, demo	Extensive AI allowed
Continuation Project (Existing product based Project)	CS, SE, IT, CY, DS, AI, CGV	Industry-ready solution, client requirements, deployment, acceptance, user docs, final report	Feature AI depends on project; APIs allowed; docs prohibited
Research Project	CS, AI, CY, DS	Literature review, proposal, experimental design, dataset, optional prototype, paper draft, report	Core/docs prohibited; AI via APIs allowed
IoT & Embedded Systems Project	CS, SE, IT	Hardware, firmware, dashboard, interface, circuit diagrams, technical docs	Core/docs prohibited; AI via APIs allowed

A.2 API representation

Each row becomes a `project_type`, versioned `ai_policy`, program applicability list, and type-specific deliverable templates. Normalized flags support validation while the original policy text remains available for human interpretation.

RSCI FYP FRAMEWORK V1.0

Appendix A. Framework traceability

A.3 Project types 6-10

Project type	Programs	Primary deliverables	AI policy normalization
Case Study Project	IT, CY, DS	Case study, current-state analysis, root-cause analysis, recommendations, roadmap	Limited core assistance; documentation prohibited
Consulting Project (AI/Data Analytics)	AI, DS	Problem analysis, data report, model/dashboard, recommendations, presentation, report	Documentation prohibited
Consulting Project (Cybersecurity)	CY, IT	Security/vulnerability reports, risk matrix, improvement plan, executive summary, report	Documentation prohibited
Consulting Project (Cloud & DevOps)	IT, CS, SE	Readiness, architecture, maturity, CI/CD, migration strategy, report	Documentation prohibited
Consulting Project (Digital Transformation)	IT, CS, SE	Maturity, process analysis, roadmap, strategy, cost-benefit, presentation, report	Documentation prohibited

A.4 Common deliverables and effort

The seed checklist includes proposal; problem/objectives; background; plan/timeline; requirements where applicable; design/methodology; implementation/investigation; validation/testing/evaluation; final report; poster/demo; oral defense; supporting artifacts; user/technical documentation; and reflection/future work. The effort module stores the framework's 9-hours-per-week and 300-hour expectations as versioned configuration.

ERROR REFERENCE

Appendix B. Error catalogue

B.1 Standard application errors

Code	HTTP	Meaning
AUTH_001	401	Invalid credentials
AUTH_002	401	Missing, expired, invalid, or revoked access token
AUTH_004	423	Account temporarily locked or disabled
AUTH_005	401	Refresh-token reuse or invalid token family
RBAC_001	403	Role or resource scope forbids operation
VAL_001	400/422	Request validation failed
WF_001	409	Illegal workflow transition
NF_001	404	Resource not found or concealed
CONFLICT_001	409	Unique or duplicate conflict
CONFLICT_002	409	Referenced resource or business invariant conflict
CONFLICT_003	409	Idempotency key reused with different payload
PRECOND_001	412	ETag or version precondition failed
FILE_001	400/413/422	Invalid, oversized, unsafe, or unscanned upload
FILE_002	404/410	File unavailable or signed link expired
AI_001	422	Incomplete or invalid AI declaration
SCORE_001	422	Invalid criterion score or incomplete sheet
SCORE_002	409	Moderation rule or limit conflict
RATE_001	429	Rate limit exceeded
SYS_001	500	Unexpected internal error
DEP_001	503	Required dependency unavailable

ERROR HANDLING

Appendix B. Error catalogue

B.2 Error detail examples

```
{
  "success":false,"data":null,
  "error":{"code":"WF_001","message":"Illegal proposal transition",
    "details":{"current_state":"APPROVED","requested_action":"submit",
               "allowed_actions":[]}},
  "meta":{"request_id":"req_01K7A"}
}

{
  "success":false,"data":null,
  "error":{"code":"VAL_001","message":"Validation failed",
    "details":[
      {"field":"criteria_scores[1].points","reason":"must be between 0 and 30"},
      {"field":"overall_comment","reason":"required for PASS_WITH_ACTIONS"}
    ]},
  "meta":{"request_id":"req_01K7B"}
}
```

B.3 Client behavior

- Display safe server messages and map known codes to contextual guidance.
- Do not retry 400, 401, 403, 404, 409, 412, or 422 without changing state/input.
- Retry 429 and transient 503 only after `Retry-After`, with jitter and a bounded attempt count.
- Preserve request_id in support reports; never include credentials or full tokens.

CONTROLLED DECISIONS

Appendix C. Changelog and open issues

C.1 Changelog

Version	Date	Change
1.0	14 October 2026	Initial implementation baseline covering portal modules, nine-role RBAC, framework traceability, examples, errors, events and release tests

C.2 Open policy and deployment decisions

ID	Decision required	Owner
OI-01	Confirm production API hostname; current base URL is a placeholder	Technical lead / IT
OI-02	Approve group sizes, supervision caps, session-state and reopen authority	Academic committee
OI-03	Approve evaluation stages, weights, aggregation, rounding, variance and moderation limits	Academic committee
OI-04	Clarify whether 300 hours and 9 hours/week apply per student or per group	Academic committee
OI-05	Approve upload types/sizes, retention, deletion, signed-URL lifetime	IT / records owner
OI-06	Approve token lifetimes, MFA scope, lockout and external-user onboarding	Security / IT
OI-07	Confirm SSO, LMS, plagiarism, email/SMS and webhook integration scope	Product owner
OI-08	Confirm exact normalized enforcement for each framework AI-policy statement	Academic integrity owner
OI-09	Approve report visibility, risk thresholds, data retention and audit access	Governance owner
OI-10	Confirm accessibility, localization, and privacy notice requirements	Product / legal

End of REST API Specification v1.0